arXiv:2605.27582v1 [cs.RO] 26 May 2026

Uni-LaViRA: Language-Vision-Robot Actions Translation for Unified Embodied Navigation

Hongyu Ding[1,2,*], Sizhuo Zhang[3,*], Ziming Xu[1,*], Jinwen Guo[1], Hongxiu Liu[1], Xingzhi Cheng[1], Zixuan Chen[1], Haifei Qi[4], Duo Wang[4], Hao Xu[1], Jieqi Shi[1,†], Yifan Zhang[2,†], Jing Huo[1,†], Jian Cheng[2], Yang Gao[1], and Jiebo Luo[5]

[1]Nanjing University, Nanjing, China [3]Beihang University, Beijing, China
[2]Institute of Automation, Chinese Academy of Sciences, Beijing, China
[4]BMW (Nanjing) Information Technology Co., Ltd., Nanjing, China
[5]University of Rochester, Rochester, NY, USA

Project page: https://xetroubadour.github.io/Uni-LaViRA/

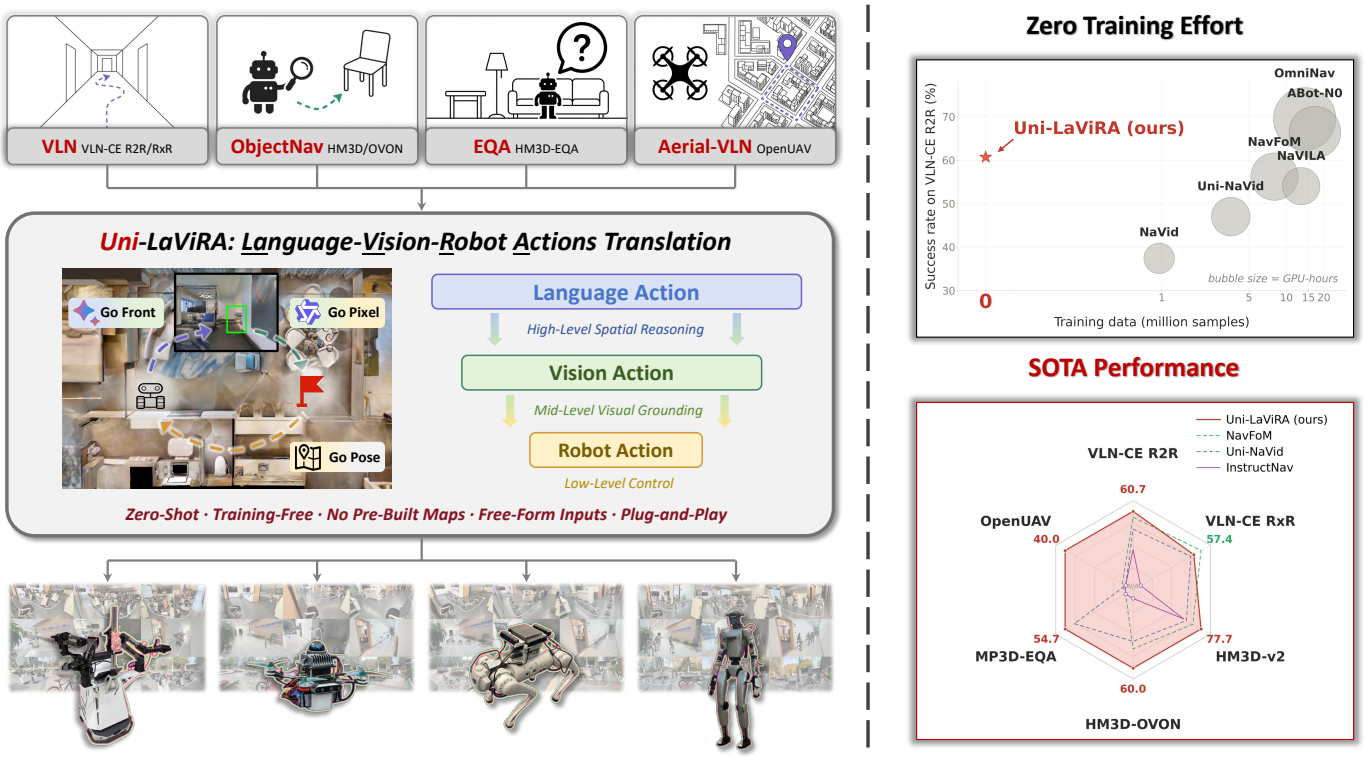


Fig. 1. **Uni-LaViRA in one picture.** (Left) A single zero-shot agentic architecture realises Language → Vision → Robot Action translation across four task families and four real embodiments. (Right) With zero training effort, Uni-LaViRA reaches state-of-the-art performance against training navigation foundation models that consume millions of samples and thousands of GPU-hours.

***Abstract*—Embodied navigation requires an agent to map language and visual observations to a stream of spatial actions that drive a real robot through environments it has never seen. The dominant approach has been to scale vision-language-action (VLA) foundation models on ever-larger collections of robot trajectories. This paper argues that, for navigation specifically, generality can be obtained structurally, not only through data scale. The underlying decision structure of navigation reduces to a single *Language-Vision-Robot Actions Translation*. The language action emits semantic-level directional command and the vision action emits a pixel-level visual target. Both outputs lie inside the natural output manifold of pretrained multimodal large language models (MLLMs), so the task can be reasoned about by an *agent* rather than learned from robot data. Therefore, we present Uni-LaViRA, a unified agentic architecture that extends the same insight to four task families (VLN-CE, ObjectNav, EQA, and Aerial-VLN) and to four heterogeneous real robots (Wheeled, Quadruped, Humanoid robot, and a self-built UAV) in a zero-shot manner. Two agent-loop mechanisms make this unification practical. TODO List Memory (TDM) rewrites a structured checklist of pending sub-goals at every step, reciting the unfinished items back into the agent's most recent attention window. Second Chance Backtrack (SCB) rolls the robot back to the pre-error state and conditions the agent's next plan on the failed sub-trajectory, turning single-pass navigation into a self-correcting process. With zero training effort, Uni-LaViRA reaches 60.7% SR on VLN-CE R2R, 51.3% on VLN-CE RxR, 77.7% on HM3D-v2, 60.0% on HM3D-OVON, 54.7% on MP3D-**

EQA, and 40.00% on OpenUAV, matching or even surpassing recent training navigation foundation models that consume millions of samples and thousands of GPU-hours.

***Index Terms*—Embodied AI, Embodied navigation, multimodal large language models, zero-shot navigation, agentic systems.**

I. Introduction

Embodied navigation, the problem of moving a physical agent through a previously unseen environment in response to a language instruction, sits at the intersection of computer vision, natural language understanding, and robotics. It has emerged as one of the central testbeds for general-purpose embodied intelligence. Its task families have expanded over the past decade. Vision-and-language navigation (VLN-CE) [1]–[3] requires an agent to follow step-by-step natural-language instructions through indoor scenes. Object-goal navigation (ObjectNav) [4]–[6] asks the agent to locate an instance of a named object category in an unfamiliar environment. Embodied question answering (EQA) [7], [8] couples navigation with active perception, requiring the agent to explore until it can answer a question grounded in the scene. Aerial vision-and-language navigation (Aerial-VLN) [9] extends instruction following to outdoor, three-dimensional flight. These tasks differ in instruction format, sensing modality, and embodiment, but share a common substrate. The agent must perceive a visual scene, interpret a language input, and emit a sequence of spatial actions that move it toward an intended state. Building a single system that generalises across this spectrum is a long-standing goal of the field.

Over the past two years, the dominant answer to generality in embodied navigation has converged on a single recipe, training ever-larger vision-language-action (VLA) foundation models on ever-growing collections of robot trajectories [10]–[14]. Each generation grows training data and compute by orders of magnitude and reports commensurate gains on performance and task coverage, in effect aiming at a scaling law for embodied navigation. This line of work stakes cross-task and cross-embodiment generalisation on data scale. This paper argues for a complementary source. For navigation specifically, generality can be obtained structurally rather than through data scale alone, once the task is decomposed in the right way. Across VLN-CE, ObjectNav, EQA, and Aerial-VLN, the underlying decision structure is stable across all four. The agent reasons about the instruction, grounds it in the current observation, and emits a spatial action. Our conference work LaViRA [15] made the central observation that this structure can be realised as a *Language-Vision-Robot Actions Translation*. The language action emits semantic-level directional command and the vision action emits a pixel-level visual target. Both outputs coincide with representations that modern multimodal large language models (MLLMs) encounter repeatedly during pretraining. Every level therefore lands inside what we call the *natural output manifold* of foundation models, and the task can be reasoned about by an agent rather than learned by a policy trained on robot data. This observation has, we believe, received less attention than it warrants from a community largely focused on scaling end-to-end navigation VLAs.

Our position can be framed as a simple question that motivates this work: does a task's action space fall inside or outside the pretrained manifold of modern MLLMs? Within embodied AI, this splits tasks into two regimes, those that are contact-rich and those that are mostly contact-free. Long-horizon dexterous manipulation sits firmly on the contact-rich side. Its action semantics, such as joint torques, contact forces, and impedance schedules, emerge from continuous physical interaction with objects. They appear as raw numerical sequences that MLLMs rarely encounter during pretraining. As a result, they sit firmly outside the MLLM manifold, and such tasks still require end-to-end VLA learning [16], [17]. Mainstream navigation methods today, by contrast, are mostly contact-free, although cluttered scenes can still involve incidental collisions or light interaction. A navigating agent does not negotiate forces with the world but moves through it, and its decisions reduce naturally to spatial reasoning in language, vision, and coordinate space. This physical property is precisely what makes the LaViRA decomposition possible, and it is what allows navigation, alone among embodied behaviours, to fully inherit the generalisation power of pretrained MLLMs without any additional training.

Beyond the unified framework itself, Uni-LaViRA introduces two mechanisms that address recurring failure modes of online agentic navigation. The first, *TODO List Memory* (TDM), reflects a key lesson from recent practice in long-horizon agent design [18]–[20]. A long-horizon agent benefits less from a longer context than from a better-attended one. Rather than feeding the agent a growing conversation history at every step, TDM maintains an explicit, dynamically updated list of the sub-tasks implied by the instruction, recording what has been completed, what remains, and what is in progress. This serves two purposes. It forces the agent to commit to a high-level plan in a structured form before it emits the next language-level action, turning each step into an explicit plan-then-act decision rather than an implicit one. And by rewriting the list at every decision step, TDM *recites* the unfinished sub-goals back into the agent's most recent attention window. The second mechanism, Second Chance Backtrack (SCB), revisits how navigation agents should handle their own mistakes. Backtracking itself is not new. SmartWay [21] introduced a backtrack action for VLN-CE, and LaViRA [15] used a basic action-level reversal. Both treat an erroneous step as something to be undone and discarded. SCB takes the opposite stance. When the agent detects a recent decision that has led the trajectory off-instruction, it rolls the robot back to the pre-error spatial state and conditions the next plan on the failed sub-trajectory. The agent then re-plans with explicit awareness of what did not work. Treating errors as informative rather than as noise turns single-pass agentic navigation into a self-correcting process.

This paper is a systematic extension of our preliminary conference version LaViRA [15], which validated the Language–Vision–Robot Actions Translation idea on a single task family

(VLN-CE). LaViRA established the three-level agentic spine, but three limitations of the conference version become bottlenecks as soon as one tries to apply the same architecture beyond instruction-following VLN. **(L1) Single-task scope.** The conference prompt schemas, action set, and stopping logic were specialised to step-by-step instruction following. A single architecture covering ObjectNav, EQA, and Aerial-VLN under one prompt interface had not been demonstrated. **(L2) Attention drift on long instructions.** On RxR, where mean instruction length is roughly four times that of R2R at about 120 words, and on multi-stage UAV flight plans, the LaViRA Language Action Model frequently forgot which intermediate sub-goals had been satisfied. The agent then emitted `stop` prematurely after the first matched landmark. **(L3) Error reasoning after a wrong decision.** LaViRA included a primitive reversal action that returned the agent to the previous waypoint, but the failed sub-trajectory was discarded, so recovery was effectively a blind retry.

Relative to the conference version, the present manuscript extends the work along the following four axes.

- **From single-task to unified embodied navigation (addressing L1).** We extend the three-level agentic framework from VLN-CE alone to four heterogeneous task families, namely VLN-CE, ObjectNav, EQA, and Aerial-VLN, under a single prompt interface, action set, and controller stack, with the Language Action Model and Vision Action Model reused verbatim across tasks.
- **TODO List Memory (TDM, addressing L2).** We introduce a structured working-memory mechanism that maintains the agent's progress as an explicit, dynamically updated list of pending and completed sub-tasks. TDM externalises the agent's plan into a form that is re-read at every decision step, so long multi-clause instructions remain fully tracked across long horizons.
- **Second Chance Backtrack (SCB, addressing L3).** We extend LaViRA's primitive reversal action into a full error-aware re-planning mechanism. Erroneous trajectories are not erased but presented back to the agent as reasoning context after the embodiment is restored to its pre-error state. This turns single-pass agentic navigation into a self-correcting process.
- **Cross-embodiment real-world deployment and extensive new analyses.** The same agentic core is deployed on four heterogeneous real robots, a wheeled bimanual Agilex Cobot Magic, a Unitree G1 humanoid, a Unitree Go1 quadruped, and a self-built quadrotor UAV, by swapping only the low-level controller. We additionally provide per-task TDM/SCB ablations, a 1,800-trial failure-mode taxonomy, and an inference-cost analysis.

Table I summarises the scope and methodological insights distinguishing the two versions. The remainder of this paper is organised as follows. Section II reviews related work on vision-and-language navigation, zero-shot navigation pipelines, navigation foundation models, and agentic AI systems. Section III presents the Uni-LaViRA framework, including the unified problem definition, the agentic architecture, the TODO List Memory and Second Chance Backtrack mechanisms, and the unified inference procedure. Section IV reports zero-shot results across the four task families, validates the 100-episode subset, compares against trained foundation models, and provides ablation studies, an inference-cost analysis, and a failure-mode taxonomy. Section V reports real-robot deployments across the four heterogeneous platforms, and Section VI concludes.

TABLE I. Scope of LaViRA and Uni-LaViRA. The latter generalises the agentic core from one task family on two embodiments to four families on four heterogeneous robots, and adds two new agent-loop mechanisms.

Aspect	LaViRA	Uni-LaViRA
Task families	1	4
Real embodiments	2	4
Working memory	Prompt history	TDM
Error recovery	1-step revert	SCB
Failure analysis	Qualitative	Quantitative + scaling

II. Related Work

A. Vision-and-Language Navigation

VLN was introduced by Anderson et al. [1] in the discrete Matterport3D graph setting [22] and later extended to continuous environments (VLN-CE) by Krantz et al. [2], with multilingual dense instructions introduced in RxR [3]. Learned methods have progressed steadily through cross-modal alignment [23], [24], explicit map representations [25]–[27], large-scale data augmentation [28], and curriculum learning [29]. The strongest end-to-end supervised methods on VLN-CE R2R are recent foundation models such as OmniNav [30] and ABot-N0 [31], which reach roughly 66–70% SR by training on heterogeneous robot data; we discuss this regime in detail in Section II-C. All of these methods rely on environment-specific training. The learned weights encode scene priors from the training distribution and do not transfer to new environments, task variations, or embodiments without costly retraining. Uni-LaViRA matches and in several cases surpasses these methods with zero robot data, indicating that scene-specific learning is not required when the action space already lies inside the natural output manifold of pretrained MLLMs.

B. Zero-Shot Navigation

Zero-shot methods avoid robot-data training but introduce other dependencies. *Waypoint-based* approaches pair an LLM with a pretrained waypoint predictor [21], [32], [33]. The predictor proposes discrete candidates and the LLM selects among them, but the pretrained predictor is itself environment-specific and its candidate set bounds where the agent can go. *Value-mapping* approaches generate a semantic heatmap from a VLM and navigate toward the peak [34]–[39]. These exploit strong VLMs for open-vocabulary grounding, but typically use the LLM only *offline* to parse the instruction. A third line

treats LLMs as planners over discrete topological graphs [40]–[42], a formulation that does not extend naturally to continuous control. Lifelong, multi-modal-goal systems such as GOAT [43], [44] contribute instance-aware semantic memory but still rely on trained skills for navigation primitives. The conference version of this work, LaViRA [15], established the three-level agentic decomposition on VLN-CE. The present paper extends it to four task families and adds the two agent-loop mechanisms developed in Sections III-C–III-D. Uni-LaViRA requires no waypoint predictor, because the Vision Action Model grounds directly on raw pixels, and keeps the Language Action Model reasoning online at every decision step in continuous environments.

C. Navigation Foundation Models

The most prominent contemporary alternative is to train a single VLA model on heterogeneous robot data and let scale do the work. Starting from VLM-based navigation on monocular video [10], subsequent foundation models pushed along orthogonal axes of unified task coverage [11], [30], action-space tokenisation for new embodiments [12], streaming inference [13], memory architectures [45], [46], and large-scale cross-embodiment training [14], [31]. Across roughly two years, training data has grown from below $1\,\mathrm{M}$ to over $16\,\mathrm{M}$ trajectories spanning quadrupeds, drones, wheeled robots, humanoids, and vehicles, while reported SR on VLN-CE R2R has climbed from below 40% to roughly 70%. The sequence reads as a coherent attempt to demonstrate a *scaling law for embodied navigation*: each generation grows training data, compute, and task coverage by orders of magnitude, with reported performance climbing in step. Uni-LaViRA takes the opposite stance. Because the decision structure of navigation already lies inside the natural output manifold of pretrained MLLMs, no training above the low-level controller is required, and each upper-level agent inherits MLLM backbone improvements directly without consuming additional robot data.

D. Agentic AI in the Physical World

The rise of agentic AI, in which foundation models reason, plan, and act through structured interaction with tools and environments [18], has reshaped thinking about AI capabilities across domains. Representative systems range from SayCan [47] and Code-as-Policies [48] to Voyager [49], Inner Monologue [20], Reflexion [19], and Agent-S [50]. Hierarchical decomposition has a long history in reinforcement learning [51], [52] and has appeared in VLN as feudal planners [53], [54].

Memory representations for long-horizon agents. MemGPT [55] treats the context window as an OS-managed virtual memory with explicit paging between a fast working set and an external store; Generative Agents [56] maintain a time-stamped memory stream with periodic reflection that summarises recent events into higher-level beliefs; MemoryBank [57] adds long-term retention with an Ebbinghaus-style forgetting schedule; and Voyager [49] grows a structured skill library as procedural memory for embodied agents. These representations are designed for general-purpose conversational or open-ended agents and store mostly free-form items; long-horizon embodied navigation specifically needs to track which sub-goals of a multi-clause instruction are still pending and which have already been satisfied at a specific scene observation. The TDM mechanism we propose in this paper sits in the same lineage but is tailored to navigation: it maintains an ordered, verifiable checklist of sub-goals that the agent re-reads and updates at every step, so completion is grounded in observations rather than left to free-form recall.

Error recovery in navigation. SmartWay [21] and LaViRA [15] both treat a failed step as something to be undone and discarded. Discarding the failed sub-trajectory, however, also discards the signal of *why* the previous decision was wrong, so the agent re-decides from the same prior with no new evidence and frequently repeats the same mistake. SCB, the second mechanism we introduce, instead conditions the re-decision on the failed sub-trajectory itself, turning error evidence into planning context. This is closer in spirit to the self-critique pattern of Reflexion [19] but operates at the spatial-trajectory level rather than the text-trial level.

III. The Uni-LaViRA Framework

Difference from LaViRA. Our method builds upon the three-level Language-Vision-Robot pipeline originally introduced in LaViRA [15]. In the present version, we extend it by scaling from one task family and two embodiments to four task families and four heterogeneous embodiments, and by adding two agent-loop mechanisms, TODO List Memory (Section III-C) and Second Chance Backtrack (Section III-D). This extension is necessary because the conference version's prompts and action set were specialised to instruction following, its working memory degraded on long-horizon directives, and its blind-retry recovery left repeated errors uncorrected, all of which become bottlenecks once the pipeline is applied to ObjectNav, EQA, and Aerial-VLN.

Section III-A abstracts the four task families into a single decision problem. Section III-B instantiates the three-level spine under the resulting unified interface. Sections III-C and III-D introduce TODO List Memory and Second Chance Backtrack, the two new agent-loop mechanisms that this paper adds to the LaViRA spine. Section III-E assembles them into a single per-episode loop.

A. Problem Definition

We consider four families of embodied navigation. The first three run inside Habitat-Sim [2]. Vision-and-language navigation in continuous environments (VLN-CE) [1], [3] covers R2R and RxR. Object-goal navigation (ObjectNav) covers HM3D-v2 and HM3D-OVON. Embodied question answering (EQA) uses MP3D-EQA [7], [8]. Aerial vision-and-language navigation (Aerial-VLN) uses OpenUAV [9] inside AirSim. Despite differing simulators, embodiments, and goal specifications, all four families share the same input and output interface from the policy's perspective.

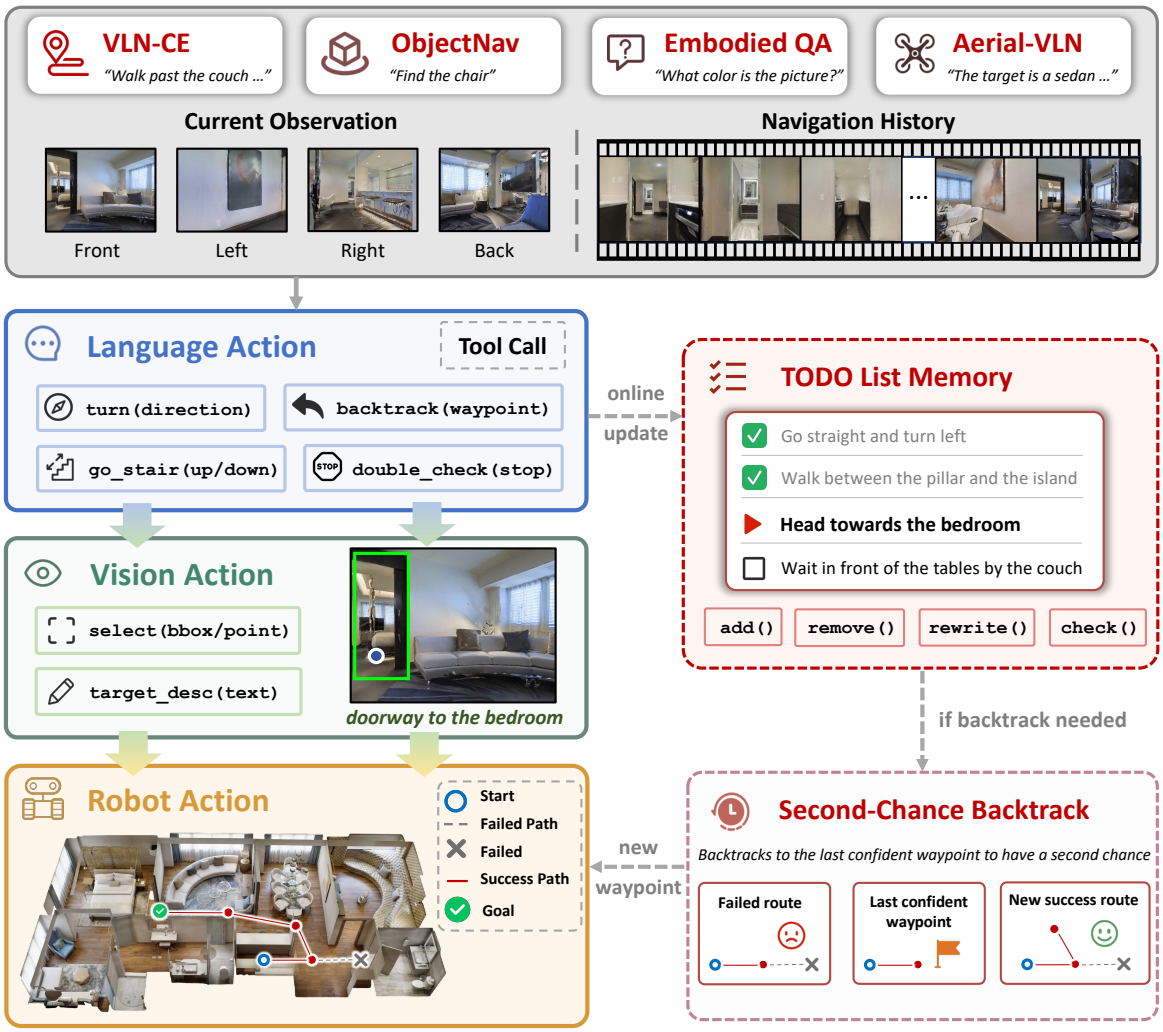


Fig. 2. **The Uni-LaViRA pipeline.** Language Action emits a tool call, Vision Action grounds it on the agent's first-person view along the chosen direction, and Robot Action dispatches it. TDM maintains an online checklist; SCB rewinds to a prior waypoint when a sub-goal fails. Together, the three-level decomposition and the two agent-loop mechanisms turn each step into a verifiable, self-correcting decision shared by all four task families.

Unified interface. At every decision step t, an agent receives

- a task specification $\mathcal{T}$ expressed in natural language. This is a route description for VLN-CE, an object category or open-vocabulary phrase for ObjectNav, a question for EQA, or a flight directive for Aerial-VLN.
- an egocentric observation $\mathcal{O}_t$ consisting of RGB and aligned depth. In simulation, ground robots carry only a single front-view RGB-D camera; the front, left, right, and back views are obtained by rotating the robot in place at each waypoint. The UAV carries five fixed cameras providing front, left, right, back, and downward views directly.
- the current pose $(x_t, y_t, z_t, \theta_t)$ in the world frame, exposed by the simulator.
- a structured history $\mathcal{H}_t$ of previously visited waypoints, keyed observations, and prior decisions.

It must emit a low-level action $\mathcal{A}_t$ in the embodiment's native action space. In simulation, ground robots in Habitat use a discrete action set (`move_forward` by 0.25 m, `turn_left`/`turn_right` by 30°, `stop`), and the UAV in AirSim uses a 3-D waypoint command (x, y, z). The corresponding native commands for real robots are described in Section V.

Output manifold. A central observation, anticipated in Section I, is that the decision relevant to $\mathcal{A}_t$ does not require regressing in the embodiment's continuous control space. At each step the task reduces to three sub-decisions. First, choose a discrete *direction* from the small set forward, left, right, back, plus a stopping decision. Second, ground that direction by selecting a *2-D pixel target* in the chosen view, from which a 3-D position is recovered via the depth channel and known intrinsics. Third, hand the resulting 3-D position to a deterministic embodiment-specific controller for short-horizon execution. Directional language and pixel-level visual targets are both objects that pretrained MLLMs already produce

reliably as text or as bounding-box coordinates. We therefore re-formulate the policy as

$$\mathcal{A}_t = \pi_{\text{robot}}\Big(\phi_{\text{vis}}\big(\phi_{\text{lang}}(\mathcal{T}, \mathcal{O}_t, \mathcal{H}_t)\big), \\ D_t,\, \mathbf{K},\, \text{pose}_t\Big), \tag{1}$$

where ϕ_{lang} is a Language Action MLLM that returns a discrete direction together with auxiliary book-keeping fields, ϕ_{vis} is a Vision Action MLLM that returns a bounding box on the selected view, D_t is the depth channel of $\mathcal{O}_t$, $\mathbf{K}$ are the camera intrinsics, and π_{robot} is a deterministic embodiment-specific controller. The factorisation in Eq. 1 is what we instantiate next.

B. Uni-LaViRA Architecture

Uni-LaViRA realises Eq. 1 as three components operating at three scales of abstraction (Fig. 2). The two upper components are general-purpose MLLMs prompted as agents, not task-specific networks.

1) Language Action: high-level planning: The Language Action Model ϕ_{lang} is a large MLLM, Gemini-3.1-Pro in our experiments, invoked at every decision step. It takes three inputs. The first is the task specification $\mathcal{T}$. The second is a four-view panoramic observation $\mathcal{O}_t = \{I_t^{\text{front}}, I_t^{\text{left}}, I_t^{\text{right}}, I_t^{\text{back}}\}$, obtained by rotating the agent in place at the current waypoint (Section III-A). The third is a structured history $\mathcal{H}_t$ encoding previously visited waypoints with brief captions, together with the sequence of past RGB observations captured along the trajectory. At every step it issues a single tool call from the discrete set

$$\mathcal{A}_t^{\text{lang}} \in \big\{\texttt{turn}(\text{DIR}),\ \texttt{backtrack}(\text{wp}_k), \\ \texttt{go_stair}(\text{UP/DOWN}),\ \texttt{double_check}(\text{STOP})\big\}, \tag{2}$$

with $\text{DIR} \in \{\text{front}, \text{left}, \text{right}, \text{back}\}$. `turn` commits to a new direction. `backtrack` returns the agent to a previously visited waypoint and triggers Second Chance Backtrack in Section III-D. `go_stair` signals a vertical transition that the controller handles with staircase-specific behaviour. `double_check` marks the current location as a candidate stopping point and asks the same MLLM to verify the goal condition before terminating the episode. Each call is wrapped in a JSON record carrying a *progress_analysis* field, an *action_reasoning* field, and, when the task uses it, the TDM updates described in Section III-C. The structured output enables fault-tolerant parsing and exposes the agent's reasoning trace for inspection.

2) Vision Action: mid-level visual grounding: The Vision Action Model ϕ_{vis} is a smaller, grounding-specialised MLLM, Qwen3.5-27B in our primary configuration. It receives the task specification $\mathcal{T}$, the progress description from the Language Action Model, and the single view I_t^{DIR} corresponding to the chosen direction. It emits a pair of tool calls,

$$\mathcal{A}_t^{\text{vis}} = \big(\texttt{select}(\text{bbox}\,/\,\text{point}),\ \texttt{target_desc}(\text{text})\big), \tag{3}$$

Here `select` returns a 2-D bounding box on the chosen view, or a pixel point when a box is ill-defined, and `target_desc` returns a short textual descriptor of the grounded target such as "doorway to the bedroom". Grounding directly on raw pixels removes the dependency on a pretrained waypoint predictor [36] and lets the agent target distant or non-traversable cues such as a hallway opening. The prompt explicitly discourages overly close targets, which suppresses the local-minimum behaviour of greedy heatmap followers.

3) Robot Action: low-level control: The Robot Action controller π_{robot} is deterministic and geometric. Given the bounding box from the Vision agent, it selects a representative pixel (u^*, v^*) inside the box, reads the depth d_t at that pixel, and back-projects to a camera-frame 3-D point $\mathbf{p}_{\text{cam}} = d_t\,\mathbf{K}^{-1}[u^*, v^*, 1]^\top$. It then transforms $\mathbf{p}_{\text{cam}}$ into the world frame using the current pose to obtain $\mathbf{p}_{\text{world}}$, plans a short-horizon trajectory from the current pose to $\mathbf{p}_{\text{world}}$ on a globally accumulated map, and executes it through the embodiment's native controller. For ground robots the map is a 2-D occupancy grid and the planner is Fast-Marching. For the UAV the map is a 3-D voxel grid and the planner is a visibility-graph search over free space. The controller is the only embodiment-specific component in the system. Cross-embodiment deployment therefore does not touch ϕ_{lang} or ϕ_{vis}.

C. TODO List Memory

The Language Action Model operates on a sliding window of recent observations. This is sufficient for short instructions but degrades on long, serial directives. RxR averages 120 words per instruction versus 29 for R2R, and OpenUAV involves multi-stage flight plans. A purely reactive agent forgets which intermediate sub-goals have already been satisfied, drifts off-route after small detours, and frequently emits `stop` prematurely after the first matched landmark. TDM addresses this gap.

Representation. TDM maintains a single ordered list $\mathcal{L}_t = [\ell_t^{(1)}, \ldots, \ell_t^{(n_t)}]$, where each item is a triple

$$\ell_t^{(i)} = (\text{content}^{(i)},\ \text{status}^{(i)},\ \text{result}^{(i)}), \tag{4}$$

with $\text{status}^{(i)} \in \{\texttt{pending}, \texttt{completed}\}$ and a free-text $\text{result}^{(i)}$ field that records the observation supporting completion.

Initialisation. At episode start, ϕ_{lang} is called once with the instruction $\mathcal{T}$ and the initial panorama $\mathcal{O}_0$ to produce $\mathcal{L}_0$ as a list of sub-goals, all marked `pending` with empty results. This single call is the only TDM-specific use of the MLLM, and all subsequent maintenance happens in the same call as the navigation decision.

Update operations. At every step t, the planning prompt of ϕ_{lang} exposes $\mathcal{L}_{t-1}$ in a compact textual form and asks the agent to emit a list of update operations U_t *before* choosing the next action. Four operations are allowed:

- `update(i, status=completed, result=r)` marks item i as completed and writes the supporting observation r.

- `rewrite(i, content=c)` refines the description of a pending item.
- `add(content=c, index=j)` introduces a new sub-goal at position j, or appends if j is omitted.
- `remove(i)` drops a sub-goal that is no longer relevant.

Items may be marked `completed` out of order, and any `completed` mark without a concrete `result` is rolled back by the parser as a verification step. The action is then chosen *conditioned on* the updated list.

$$U_t = \phi_{\text{lang}}^{\text{todo}}(\mathcal{T}, \mathcal{O}_t, \mathcal{H}_t, \mathcal{L}_{t-1}), \tag{5}$$

$$\mathcal{L}_t = \text{apply}(\mathcal{L}_{t-1}, U_t), \tag{6}$$

$$\mathcal{A}_t^{\text{lang}} = \phi_{\text{lang}}^{\text{act}}(\mathcal{T}, \mathcal{O}_t, \mathcal{H}_t, \mathcal{L}_t). \tag{7}$$

In practice, $\phi_{\text{lang}}^{\text{todo}}$ and $\phi_{\text{lang}}^{\text{act}}$ are emitted by a single MLLM call with separate `reasoning_todo` and `reasoning_action` fields, which empirically prevents the two reasoning modes from interfering.

Prompt-only implementation. TDM is implemented entirely in prompt space. No parameters are introduced and no task-specific training is performed. The list is structured persistent scratch memory that the same MLLM produces and consumes. This matches the output-manifold premise of Section III-A. The long horizon is absorbed by the natural language a foundation model already writes well, not by the trajectory regression a foundation model writes poorly.

D. Second Chance Backtrack

A single forward decision is brittle in cluttered indoor scenes. ϕ_{lang} may pick a corridor that visually looks correct but turns out to be a dead end, or ϕ_{vis} may ground onto a distractor that resembles the target class. A monolithic policy that treats every step independently has no recourse from such failures. SCB makes recovery a first-class action by combining two ingredients. The first is backtracking on the global map, and the second is a re-decision conditioned on the *evidence of the failure*.

Backtrack as an explicit action. Throughout an episode, Uni-LaViRA tags every location at which ϕ_{lang} is invoked as a waypoint on the globally accumulated occupancy map and stores it in a fixed-size buffer. At every step, ϕ_{lang}'s discrete action set therefore includes $\texttt{backtrack}(\text{wp}_k)$ for any previously visited waypoint k. Selecting wp_k triggers the Robot Action controller to plan an FMM path from the current pose to wp_k on the current map. Because the map accumulates obstacles seen since the original visit, the return path is consistent with the latest scene geometry, not with the agent's earlier and possibly incorrect mental model.

The second-chance re-decision. Returning to a waypoint is not enough. The agent must avoid repeating the failed choice. Upon arrival at wp_k, Uni-LaViRA issues a dedicated re-planning call to ϕ_{lang} with three pieces of evidence explicitly assembled.

- the original task specification $\mathcal{T}$ and the panorama $\mathcal{O}^{(\text{wp}_k)}$ at the waypoint.
- the failed direction $\mathcal{A}_{\text{prev}}^{\text{lang}} \in \{\text{front}, \text{left}, \text{right}, \text{back}\}$ that was selected when the agent first left wp_k.
- the egocentric trajectory image sequence Π_k^{fail} collected between wp_k and the dead end that triggered backtracking.

The re-planning prompt instructs the MLLM to inspect Π_k^{fail}, diagnose why $\mathcal{A}_{\text{prev}}^{\text{lang}}$ failed, and choose a new direction $\mathcal{A}_{\text{new}}^{\text{lang}}$ from the remaining set. The re-decision is therefore conditioned on the same observations that produced the failure, not on a blind exclusion of the failed direction. Once $\mathcal{A}_{\text{new}}^{\text{lang}}$ is fixed, ϕ_{vis} is re-invoked on $I_t^{\mathcal{A}_{\text{new}}^{\text{lang}}}$ to produce a fresh bounding box and the controller resumes normal execution.

Compatibility with TDM. SCB is orthogonal to TDM. The TODO list survives backtracking, and the re-decision call is allowed to emit TDM updates that mark the failed sub-goal as still `pending` or to rewrite it with a sharper description. This keeps the agent's high-level plan in sync with the recovery action.

Cross-embodiment. Because SCB is a property of the agent loop, not of the embodiment, it transfers to all four robots without modification. For the UAV, the 2-D occupancy map is replaced by a 3-D voxel grid and the FMM step becomes a visibility-graph plan to wp_k in 3-D. The rest of the procedure is identical.

Algorithm 1 One episode of Uni-LaViRA.

Require: Task spec $\mathcal{T}$, initial observation $\mathcal{O}_0$, MLLMs $\phi_{\text{lang}}, \phi_{\text{vis}}$, controller π_{robot}

1: $\mathcal{L}_0 \leftarrow \phi_{\text{lang}}^{\text{init}}(\mathcal{T}, \mathcal{O}_0)$ // TDM init
2: $\mathcal{H}_0 \leftarrow \emptyset$, $\text{WP} \leftarrow \{(\text{wp}_0, \mathcal{O}_0)\}$
3: **for** $t = 1, 2, \ldots, T_{\max}$ **do**
4: $(U_t, \mathcal{A}_t^{\text{lang}}) \leftarrow \phi_{\text{lang}}(\mathcal{T}, \mathcal{O}_t, \mathcal{H}_t, \mathcal{L}_{t-1})$
5: $\mathcal{L}_t \leftarrow \text{apply}(\mathcal{L}_{t-1}, U_t)$ // TDM update
6: **if** $\mathcal{A}_t^{\text{lang}} = \texttt{stop}$ **then**
7: **break**
8: **else if** $\mathcal{A}_t^{\text{lang}} = \texttt{backtrack}(\text{wp}_k)$ **then**
9: $\pi_{\text{robot}}.\text{plan_to}(\text{wp}_k)$
10: $\mathcal{A}_t^{\text{lang}} \leftarrow \phi_{\text{lang}}^{\text{recover}}(\mathcal{T}, \mathcal{O}^{(\text{wp}_k)}, \Pi_k^{\text{fail}})$ // SCB
11: **end if**
12: $\mathcal{A}_t^{\text{vis}} \leftarrow \phi_{\text{vis}}(\mathcal{T}, \mathcal{A}_t^{\text{lang}}, I_t^{\text{DIR}})$
13: $\mathcal{A}_t \leftarrow \pi_{\text{robot}}(\mathcal{A}_t^{\text{vis}}, D_t, \mathbf{K}, \text{pose}_t)$
14: Execute $\mathcal{A}_t$; update $\mathcal{O}_{t+1}, \mathcal{H}_{t+1}, \text{WP}$
15: **end for**

E. Unified Inference Procedure

Algorithm 1 summarises one episode of Uni-LaViRA. Lines 1–2 initialise the TDM list and the visited-waypoint history. Lines 4–5 query the Language Action Model, which jointly emits TDM update operations and an action choice, after which the list is refreshed. Lines 6–7 handle episode termination on `stop`. Lines 8–10 implement SCB: on a `backtrack` action, the controller rewinds the embodiment to the chosen prior waypoint and re-prompts the planner with the failed sub-trajectory as evidence. Lines 12–14 instantiate the Vision Action and Robot Action levels and execute the

TABLE II. **100-episode subset vs. original full-set reports.** For each task we reproduce two public baselines on our stratified 100-episode subset (**Sub.**; mean±std over three seeds when the baseline is stochastic) and align them against the value the original paper reports on the full val-unseen split (**Full**). Dashes mark metrics that are not standard for the task. Most cells agree within ±2 absolute points; see Section IV-B.

		NE↓		OSR↑		SPL↑		SR↑		ACC↑	
Task	**Method**	Sub.	Full	Sub.	Full	Sub.	Full	Sub.	Full	Sub.	Full
VLN-CE R2R	Uni-NaVid [11]	5.09±0.16	5.58	52.7±0.9	53.3	42.8±1.6	42.7	46.7±2.1	47.0	–	–
	StreamVLN [13]	5.70	4.98	60.0	64.2	48.4	51.9	56.0	56.9	–	–
VLN-CE RxR	Uni-NaVid [11]	6.58±0.51	6.24	56.3±1.3	55.5	39.4±1.4	40.9	47.7±1.7	48.7	–	–
	StreamVLN [13]	6.48	6.22	61.0	61.9	47.3	46.0	54.0	52.9	–	–
HM3D-v2	VLFM [36]	–	–	–	–	31.1	31.0	63.0	62.6	–	–
	ApexNav [58]	–	–	–	–	34.5±0.5	38.0	76.7±0.6	76.2	–	–
HM3D-OVON	VLFM [36]	–	–	–	–	18.3	22.2	36.0	38.5	–	–
	MTU3D [59]	–	–	–	–	14.2±0.3	12.1	40.7±3.2	40.8	–	–
MP3D-EQA	EQA [7]	–	–	–	–	–	–	–	–	43.0	46.0
	Uni-NaVid [11]	–	–	–	–	–	–	–	–	46.0±1.6	47.3
OpenUAV	TravelUAV [9]	130.7±7.0	139.0	5.6±4.6	20.8	2.6±1.9	3.8	3.0±2.0	4.2	–	–
	AerialVLA [60]	77.6±9.1	67.4	49.0±2.7	52.9	25.9±2.2	28.2	33.3±2.1	37.6	–	–

resulting low-level action. No part of the loop touches a learned parameter.

IV. Simulation Experiments

We evaluate Uni-LaViRA on six standard benchmarks spanning four task families: VLN-CE R2R [2] and RxR [3], HM3D-v2 [5], [61], HM3D-OVON [6], MP3D-EQA [7], [8], and OpenUAV [9]. The same agentic framework is used across all six benchmarks with no task-specific training and no parameter update.

A. Experimental Setup

Simulators and observations. For VLN-CE, ObjectNav, and EQA we use the Habitat simulator [62]. HM3D-v2 and HM3D-OVON run on the Habitat-Matterport 3D scenes [5], [61], while R2R, RxR, and MP3D-EQA run on the Matterport3D scenes [22]. For OpenUAV, we use the official AirSim-based OpenUAV dataset [9], which contains photorealistic outdoor scenes, including cities, towns, forests, and other environments. Ground robots carry only a single front-view 640×480 RGB-D camera in Habitat; the front, left, right, and back views are obtained by rotating the robot in place at each waypoint. The UAV in AirSim carries five fixed RGB-D cameras providing front, left, right, back, and downward views directly.

Agent backbones. The Language Action Model is Gemini-3.1-Pro and the Vision Action Model is Qwen3.5-27B. Neither agent is fine-tuned, and both are queried in pure inference mode through their public APIs. Low-level path planning uses Fast-Marching on a globally accumulated 2-D occupancy map. For the UAV, the generated 3-D waypoints are executed directly by the AirSim flight controller. Simulator rollouts run on 8 NVIDIA RTX 4090 GPUs in parallel.

Evaluation protocol. For every benchmark we evaluate on the corresponding *val-unseen* split, or the *UM* unseen-map test split for OpenUAV. To control evaluation cost while preserving representativeness, we draw a stratified 100-episode subset from each val-unseen pool, following the common protocol adopted by prior zero-shot VLN work [10], [32]. We stratify on instruction and trajectory length for R2R/RxR, target category for HM3D-v2, semantic class for HM3D-OVON, question type for MP3D-EQA, and Easy/Hard path type for OpenUAV. MLLM responses are stochastic, so we run **three independent seeds** per benchmark and report **mean ± standard deviation**.

OpenUAV-specific implementation. The base agentic stack needs two adaptations for OpenUAV. We widen the panorama to five views by adding a downward camera, because aerial targets sit above or below the flight path as often as forward. We also add a *direction-constrained module*, which kicks in when the Vision Action Model fails to ground the target. The Language Action Model then reasons explicitly about the initial pose, the current pose, and the global goal direction, with the depth channel supplying obstacle-distance estimates for safe motion. The module is a prompt-level addition to ϕ_{lang} and introduces no new parameters.

Metrics. We follow the standard navigation metrics [1]: Navigation Error (NE), Success Rate (SR), Oracle Success Rate (OSR), Success-weighted Path Length (SPL), and nDTW for RxR. Success uses the standard 3m threshold for VLN-CE and the task-specific radius for HM3D-v2 and HM3D-OVON. For EQA, ACC requires the answer to match the ground truth under the OpenEQA LLM-judge protocol [8]. For OpenUAV we use the benchmark's metrics.

B. Validating the 100-Episode Subset

To check that our stratified 100-episode subsets are faithful proxies for the full val-unseen splits, we reproduce two strong public baselines per task on our subset and compare against the values their original papers report on the full split. Stochastic baselines are run with three seeds, deterministic ones once.

Across the 34 metric cells with a published full-split reference, 21 (62%) agree within ±2 absolute points and 31 (91%) within ±5. The three residuals beyond ±5 all fall on OpenUAV: TravelUAV NE (130.7 vs. 139.0), TravelUAV OSR (5.6 vs. 20.8), and AerialVLA NE (77.6 vs. 67.4). These are the cells with the largest dynamic range in the table, since

TABLE III. **Main results on VLN-CE R2R and RxR val-unseen.** Our numbers are mean ± std over three seeds. **Best** and second-best per column within the zero-shot block.

Method	VLN-CE R2R Val-Unseen				VLN-CE RxR Val-Unseen			
	NE↓	OSR↑	SR↑	SPL↑	NE↓	SR↑	SPL↑	nDTW↑
Supervised Learning (Training Method)								
NaVid [10]	5.47	49.1	37.4	35.9	8.41	34.5	23.8	–
Uni-NaVid [11]	5.58	53.3	47.0	42.7	6.24	48.7	40.9	–
NaVILA [12]	5.22	62.5	54.0	49.0	6.77	49.3	44.0	58.8
StreamVLN [13]	4.98	64.2	56.9	51.9	6.22	52.9	46.0	61.9
NavFoM [14]	5.01	64.9	56.2	51.2	5.51	57.4	49.4	60.2
JanusVLN [45]	4.78	65.2	60.5	56.8	6.06	56.2	47.5	62.1
OmniNav [30]	3.74	74.6	69.5	66.1	3.77	73.6	62.0	–
InternVLA-N1 [46]	4.83	63.3	58.2	54.0	5.91	53.5	46.1	65.3
ABot-N0 [31]	3.78	70.8	66.4	63.9	3.83	69.3	60.0	–
SPAN-Nav [63]	4.07	75.3	66.3	59.3	4.20	69.7	60.1	67.9
Zero-Shot (Training-Free)								
InstructNav [35]	6.89	47.0	31.0	24.0	–	–	–	–
Open-Nav [32]	6.70	23.0	19.0	16.1	–	–	–	–
CA-Nav [34]	7.58	48.0	25.3	10.8	10.4	19.0	6.0	13.5
SmartWay [21]	7.01	51.0	29.0	22.5	–	–	–	–
GC-VLN [37]	7.30	41.8	33.6	16.3	8.80	33.8	13.8	–
LaViRA [15]	6.54±0.27	48.7±2.1	38.3±0.6	28.3±0.9	–	–	–	–
HiMemVLN [64]	6.65	36.0	30.0	26.9	–	–	–	–
Three-Step Nav [65]	5.87	39.0	34.0	29.1	9.21	22.0	16.1	45.7
Uni-LaViRA (Ours)	**3.66**±0.26	**73.7**±1.2	**60.7**±2.1	**47.7**±1.6	**6.48**±0.14	**51.3**±2.3	**34.0**±1.9	**53.7**±0.6

TABLE IV. **Main results on OpenUAV `UM` (Test-Unseen-Map).** Easy/Hard are the standard path-type subsets; Full is their union. Uni-LaViRA is mean ± std over three seeds and is the first training-free entry on this benchmark. Bold marks the best in the zero-shot block.

Method	Full				Easy				Hard			
	NE↓	OSR↑	SR↑	SPL↑	NE↓	OSR↑	SR↑	SPL↑	NE↓	OSR↑	SR↑	SPL↑
Supervised Learning (Training Method)												
Random Action [9]	203	0.00	0.00	0.00	158	0.00	0.00	0.00	266	0.00	0.00	0.00
Fixed Action [9]	180	2.61	0.52	0.39	133	4.28	0.89	0.67	248	0.25	0.00	0.00
CMA [9]	142	10.0	2.30	2.16	102	14.3	3.57	3.33	197	4.03	0.50	0.50
TravelUAV [9]	139	20.8	4.18	3.84	103	22.8	4.63	4.24	189	17.9	3.53	3.28
NavFoM [14]	125	19.0	6.30	5.68	102	20.1	6.77	6.04	171	15.7	5.36	4.97
LongFly [66]	108	30.3	11.3	9.32	78.6	34.3	13.0	10.3	148	24.9	9.02	7.98
AerialVLA [60]	67.4	52.9	37.6	28.2	45.0	58.5	41.9	29.7	99.1	45.1	31.5	26.1
SpatialFly [67]	104	30.4	13.6	11.1	72.9	34.3	16.1	12.7	146	25.1	10.2	9.04
Zero-Shot (Training-Free)												
Uni-LaViRA (Ours)	**84.29**±11.20	**67.33**±5.51	**40.00**±3.46	**30.37**±2.23	**77.05**±7.40	**68.75**±3.61	**39.58**±2.09	**27.95**±3.18	**90.96**±14.88	**66.03**±7.83	**40.38**±8.21	**32.60**±5.94

OpenUAV trajectories span hundreds of metres and TravelUAV succeeds on only a few percent of episodes even on the full split, making derived metrics intrinsically high-variance under 100-episode resampling. In other words, the baseline numbers we reproduce on the 100-episode subset closely track the values reported on the corresponding full val-unseen splits, so the subset preserves the difficulty profile of the official splits rather than introducing a separate evaluation regime. We therefore use the 100-episode subset as a faithful evaluation proxy for the comparisons in Section IV-C. Subset SR values also sit at or below their full-split counterparts on four of the six tasks (R2R, HM3D-OVON, MP3D-EQA, OpenUAV) and within 1.1 absolute points on RxR, indicating that the stratified subset is not artificially easier than the official splits.

C. Analysis of Main Results

Cross-task headline. A single zero-shot, training-free agent reaches SR of 60.7, 51.3, 77.7, 60.0, 54.7, and 40.0 on R2R, RxR, HM3D-v2, HM3D-OVON, MP3D-EQA, and OpenUAV-UM Full respectively (full numbers in Tables III–V). Uni-LaViRA is the strongest training-free method on all six benchmarks, and on four of them—HM3D-v2, HM3D-OVON, MP3D-EQA, and OpenUAV—it also surpasses the best reported *trained* foundation model despite using no robot trajectories. The two VLN-CE tasks are the only cases where the trained group still leads, driven by recent multi-task VLAs; per-task margins are analysed below.

Vision-and-language navigation. On R2R val-unseen, Uni-LaViRA attains 60.7% SR with no robot-data training. It beats the strongest zero-shot baseline LaViRA at 38.3% by 22.4 points, matches JanusVLN at 60.5%, and surpasses NavFoM at 56.2% which used ∼8 M trajectories by 4.5 points. Its NE of 3.66 m is the lowest in both groups. Only the larger multi-task VLAs OmniNav at 69.5%, ABot-N0 at 66.4%, and SPAN-Nav at 66.3% report higher R2R SR. On RxR, where the mean instruction length is ∼120 words, Uni-LaViRA reaches 51.3± 2.3% SR, comparable to NaVILA at 49.3%. The strongest trained VLAs NavFoM and JanusVLN remain ahead by 5–6 SR points at 57.4% and 56.2%, and OmniNav, ABot-N0, and SPAN-Nav are further ahead. RxR SPL of 34.0 trails the trained-VLA group because agentic exploration samples next directions under uncertainty rather than following a learned shortcut.

TABLE V. **Main results on HM3D-v2, HM3D-OVON, and MP3D-EQA.** Uni-LaViRA is mean ± std over three seeds. **Best** and second-best per column within the zero-shot block.

Method	HM3D-v2 SR↑	HM3D-v2 SPL↑	HM3D-OVON SR↑	HM3D-OVON SPL↑	EQA ACC↑
Supervised Learning (Training Method)					
EQA(w/GT) [7]	–	–	–	–	46.0
DD-PPO [68]	27.9	14.2	–	–	–
Habitat-Web [69]	57.6	23.8	–	–	–
PIRLNav [70]	70.4	34.1	–	–	–
OVRL-v2 [71]	64.7	28.1	–	–	–
NaviLLM [72]	–	–	–	–	44.5
DAgRL+OD [6]	–	–	37.1	19.9	–
Uni-NaVid [11]	73.7	37.1	39.5	19.8	47.3
NavFoM [14]	–	–	43.6	31.3	–
FiLM-Nav [73]	77.0	41.3	40.8	24.4	–
OmniNav [30]	–	–	59.2	33.2	–
ABot-N0 [31]	–	–	54.0	30.5	–
Zero-Shot (Training-Free)					
L3MVN [74]	54.2	25.5	–	–	–
VLFM [36]	62.6	31.0	38.5	22.2	–
OpenFMNav [39]	54.9	24.4	–	–	–
InstructNav [35]	58.0	20.9	–	–	–
SG-Nav [75]	54.0	24.9	–	–	–
ApexNav [58]	76.2	38.0	–	–	–
DSCD-Nav [76]	73.0	38.7	–	–	–
ReMemNav [77]	67.8	36.6	–	–	–
Uni-LaViRA (Ours)	**77.7**±2.1	**46.1**±1.8	**60.0**±6.1	**40.5**±1.1	**54.7**±2.3

Object-goal and open-vocabulary navigation. Uni-LaViRA performs strongly on both HM3D-v2 and HM3D-OVON. On HM3D-v2 it reaches 77.7**% SR**, narrowly ahead of the strongest trained baseline FiLM-Nav at 77.0% while improving its SPL by 4.8 points, and clear of the trained VLA Uni-NaVid at 73.7% and OpenFMNav by 22.8 points. On HM3D-OVON it reaches 60.0**% SR**, beating OmniNav by 0.8 points and ABot-N0 by 6.0 points. Two properties of the agentic stack contribute. First, the Language Action Model reasons over room-level semantic priors that an MLLM has acquired at pretraining, which value-map baselines cannot exploit because they consume the VLM only as a per-pixel heatmap. Second, the Language Action Model and the Vision Action Model both take free-form natural-language input, so the open-vocabulary HM3D-OVON setting is supported without any modification to the pipeline.

Embodied question answering. On MP3D-EQA, Uni-LaViRA attains $54.7 \pm 2.3\%$ answer-correctness ACC, ahead of the strongest trained baseline Uni-NaVid at 47.3% by 7.4 points. Most failures occur at the final question-answering step rather than during navigation. The agent reaches the right area but the MLLM misidentifies fine-grained visual details such as confusing dark brown with black for furniture colour. The EQA failure mode is inherited from the Language Action backbone and is removed simply by upgrading it.

UAV vision-and-language navigation. On the OpenUAV UM split, Uni-LaViRA attains $40.00 \pm 3.46\%$ Full SR and $30.37 \pm 2.23\%$ SPL with no UAV-specific training. This surpasses NavFoM by +33.7 SR and +24.7 SPL, runs roughly nine times the SR of TravelUAV, and exceeds the strongest reported trained UAV-VLA AerialVLA at 37.6% SR and 28.2% SPL. UAV-specialised baselines LongFly at 11.3% and SpatialFly at 13.6% sit far below both Uni-LaViRA and AerialVLA, indicating that geometric-prior fine-tuning does not close the gap to a Language → Vision agentic pipeline. AerialVLA holds a small +2.3 SR lead on the Easy split where its end-to-end controller benefits from short, straight trajectories. On the Hard split, with trajectories above 250 m, sparse semantic landmarks, and the target often out of view, Uni-LaViRA opens a +8.9 SR and +6.5 SPL gap, indicating that MLLM-driven long-range decision-making becomes the dominant factor when instructions are long and 3-D scene reasoning is required. The 67.3% OSR shows the agent reaches the goal vicinity in roughly two-thirds of episodes. The 27-point OSR–SR gap reflects high-altitude flight, where the UAV may pass over the target without clearly observing ground-level cues and fails to recognise successful arrival.

D. Ablation Studies

We ablate the two agentic mechanisms introduced in Sections III-C and III-D, namely *TODO List Memory* (TDM) and *Second Chance Backtrack* (SCB), across all six benchmarks. Four configurations are compared. *Full* enables both mechanisms. *w/o TDM* keeps only SCB, *w/o SCB* keeps only TDM, and *w/o both* disables both. Each configuration is run three times with different sampling seeds. Table VI reports SR mean ± std.

Both mechanisms are needed. The Full configuration outperforms every single-mechanism and no-mechanism configuration on every benchmark. Disabling either mechanism alone leaves SR within a few points of disabling both. On R2R, for example, all three reduced configurations sit in the 49–50% band while Full reaches 60.7%. This reveals a strong synergy. TDM keeps the Language Action Model committed to a coherent multi-step plan, and SCB lets it recover when grounding errors push it off that plan. Either mechanism on its own is brittle. TDM commits the agent to a plan it cannot correct, and SCB lets it correct without a stable target to correct *toward*.

Per-task contribution profile. The marginal contributions Δ_{TDM} and Δ_{SCB} tell a task-dependent story. TDM dominates on the long-horizon and answer-driven tasks at +11.4 on R2R, +6.6 on RxR, +6.0 on MP3D-EQA, and +5.67 on OpenUAV, where instructions or queries require the agent to maintain explicit checkpoints across many steps. SCB dominates on the indoor object-search benchmarks at +9.0 on HM3D-v2 and +5.3 on HM3D-OVON, where the agent must explore many candidate rooms and frequently encounters distractor grounding, so flexible backtracking is the main lever for recovery. On OpenUAV the contrast is particularly sharp. $\Delta_{\mathrm{TDM}} = +5.67$ on Full SR widens to roughly +10 SR points on the Hard subset with trajectories above 250 m, where the Language Action Model must hold a goal direction across many out-of-view steps without drifting toward locally salient distractors. SCB, in contrast, contributes only +2.00 SR points on OpenUAV and is triggered in a small minority

TABLE VI. **Ablation of TDM and SCB across six benchmarks.** Cells are SR mean ± std over three seeds, in percent. MP3D-EQA reports ACC in place of SR. OpenUAV uses the Full split. The last column reports the unweighted mean across the six benchmarks. Δ rows give the marginal contribution of each mechanism with the other held on. Bold marks the strongest configuration per column.

Configuration	R2R	RxR	HM3D-v2	HM3D-OVON	EQA	OpenUAV	Avg
Full	**60.7±2.1**	**51.3±2.3**	**77.7±2.1**	**60.0±6.1**	**54.7±2.3**	**40.00±3.46**	**57.4**
w/o TDM	49.3±1.5	44.7±3.6	73.7±2.5	57.3±4.5	48.7±0.6	34.33±2.31	51.3
w/o SCB	50.0±2.0	44.3±3.8	68.7±0.6	54.7±3.8	51.7±1.5	38.00±3.61	51.2
w/o both	49.0±4.6	40.0±4.2	66.3±1.5	52.0±3.0	50.3±4.0	33.67±3.51	48.6
Δ_{TDM} = Full − w/o TDM	+11.4	+6.6	+4.0	+2.7	+6.0	+5.67	+6.1
Δ_{SCB} = Full − w/o SCB	+10.7	+7.0	+9.0	+5.3	+3.0	+2.00	+6.2

TABLE VII. **Compute trade-off: Uni-LaViRA turns a fixed training cost into a marginal per-call one.** Training cost is the up-front spend reported in each source paper. Inference cost is measured per MLLM call on our 1,800-trajectory Full-configuration pool.

Method	Training		Inference / call	
	#Training Samples	GPU-hours	Tokens in+out	FLOPs
Supervised Learning (Training Method)				
NaVid [10]	0.95 M	320	–	–
Uni-NaVid [11]	3.6 M	1,400	–	–
NaVILA [12]	13.1 M	1,152	–	–
NavFoM [14]	8.0 M	4,032	–	–
OmniNav [30]	14.0 M	14,592	–	–
ABot-N0 [31]	16.9 M	6,144	–	–
Zero-Shot (Training-Free)				
LaViRA-LA [15]	0	0	3.9k+0.2k	∼1.65P
LaViRA-VA [15]	0	0	1.0k+0.07k	∼69T
Uni-LaViRA-LA	0	0	22.3k+1.6k	∼9.6P
Uni-LaViRA-VA	0	0	0.99k+0.11k	∼59T

of episodes, acting as a recovery mechanism for individual local failures rather than as a persistent driver of performance. On R2R and RxR the two mechanisms are closely matched ($\Delta_{\text{TDM}}=+11.4$ vs $\Delta_{\text{SCB}}=+10.7$ on R2R; $\Delta_{\text{TDM}}=+6.6$ vs $\Delta_{\text{SCB}}=+7.0$ on RxR), reflecting the combination of long instructions and cluttered multi-room scenes that exercises both checklist maintenance and backtracking. This pattern matches the design intent of Sections III-C and III-D. TDM addresses attention drift on long horizons, SCB addresses error recovery in cluttered scenes, and the two are complementary rather than redundant.

E. Inference Cost and Training Cost

This section makes one point: Uni-LaViRA replaces a fixed, up-front training cost with a marginal cost paid per evaluation. Training at NavFoM scale [14] needs a 56-H100 cluster running uninterrupted for several days, hardware that most universities and independent labs simply cannot access. Moving the spend to inference puts foundation model navigation research within reach of any group with API access.

Beyond accessibility, the two regimes can be compared head to head in raw FLOPs. NavFoM reports fine-tuning on 56 NVIDIA H100 GPUs for ∼72 hours, a total of 4,032 H100-hours. At a realistic 40% MFU on H100 BF16 dense throughput of ∼989 TFLOPS[1], the training run consumes ∼5.7 ZFLOPs (5.7×10^{21} FLOPs), equivalent to roughly 570 full six-task Uni-LaViRA evaluations (each ∼10 EFLOPs). A typical research project runs at most a few dozen full six-task evaluations, well below this 570 break-even, so in any realistic budget Uni-LaViRA is also cheaper in cumulative compute, not only more accessible.

The per-call numbers in Table VII come from the same Full configuration as Section IV-C, with Gemini-3.1-Pro for LA and Qwen3.5-27B for VA, measured over the 1,800-trajectory pool. Per call, LA averages 22,305 input and 1,618 output tokens, of which 88.7% are reasoning-trace tokens, while VA averages ∼990 input and ∼110 output tokens with no reasoning trace. Per-call compute works out to ∼9.6 PFLOPs for LA and ∼59 TFLOPs for VA, as listed in Table VII. Of the ∼10 EFLOPs consumed by a six-task Full evaluation, 93.2% goes to LA prefill, 6.0% to LA reasoning tokens, 0.8% to LA visible output, and under 0.1% to VA, so input-context length is the dominant lever. In wall-clock terms, a complete six-task 600-episode run finishes in 5–7 hours on 40 parallel API workers, with a single LA call taking 30–60 seconds when the API is healthy and 80–150 seconds under contention, dominated by network round-trip and reasoning-token decode, while the local planner and controller each run in under 10 ms.

F. Qualitative Analysis and Failure Modes

Figure 3 shows two patterns in every row. Each `NAV` keyframe is a grounded sub-goal phrase such as "open arched doorway on the forward" or "doorway leading to the hallway on the forward", so the Language Action Model commits in object- and region-level terms rather than as raw waypoints. The explicit `BACKTRACK` action surfaces mid-trajectory in five of six rows, including the EQA episode where the agent first considers an oven, retreats to a previous waypoint, then locks the correct view and answers "silver". This matches the SCB ablation in Table VI. Backtracking is routine rather than a rare recovery.

We pool all 1,800 trials across the six benchmarks and classify failures from the final-state metrics *success*, *oracle_success*, *distance_to_goal*, and *path_length*, using a small set of deterministic geometric rules. Table VIII reports the resulting categories.

[1]Per the NVIDIA H100 SXM5 datasheet, BF16 Tensor-Core throughput without sparsity.



Fig. 3. **Trajectory visualization across the six simulation benchmarks.** Each row shows five keyframes from a representative successful episode, annotated with the literal Language Action emitted by the agent and the corresponding Vision Action grounding it on the first-person view.

Early stop on wrong target. This mode accounts for 17.7% of trials and 45.8% of failures. The agent triggers `STOP` after a short trajectory of $\bar{\ell}$=11.9 m and $\bar{t}$=217 steps, ending ∼11 m from the goal. The Language Action either misread the

TABLE VIII. **Failure-mode breakdown across** 1,800 **trials.** Failures are categorised post-hoc from final-state geometry, pooled across the six benchmarks. The top three categories account for ∼90% of all failures; the two geometric tail modes split the remainder.

Failure Category	n	% all trials	% of failures	Dist. to goal (m)	Path length (m)	Steps taken
Early stop on wrong target	319	17.7	45.8	11.0±4.73	11.9±6.9	217±107
Reached but missed STOP	172	9.6	24.7	3.72±2.25	14.3±10.3	237±149
Wrong answer (EQA)	136	7.6	19.5	–	–	–
Approached but undershot	48	2.7	6.9	4.13±0.57	9.6±7.3	170±119
Wandered and lost	22	1.2	3.2	9.49±4.26	30.2±4.5	440±92

instruction, locked onto the wrong landmark, or over-trusted a VA bounding box on a distractor. This row dominates RxR at 34% and R2R at 26%, where long instructions make grounded landmarks easy to confuse.

Reached but missed STOP. This mode covers 9.6% of trials and 24.7% of failures. $oracle_success{=}1$ but the agent walks away from the goal after passing the 3 m success ring. Trajectories are longer at $\bar{\ell}{=}14.3$ m and $\bar{t}{=}237$ steps, with a final distance of 3.7 m. HM3D-OVON at 17% and HM3D-v2 at 10% are most affected, consistent with their open-vocabulary targets being easier to over-shoot.

Wrong answer on EQA. This mode is 7.6% of trials and 19.5% of failures, all on MP3D-EQA. Navigation may have been adequate but the LA's final-view answer is wrong, a failure downstream of the Language backbone's fine-grained visual reasoning.

Geometric tail. *Approached but undershot* contributes 2.7% of trials and *Wandered and lost* 1.2%. Together they cover 10.0% of failures and under 4 SR points. These are Robot-Action errors from depth artefacts, FMM dead-ends, and infinite-loop traversals.

V. Real-World Deployment

We deploy Uni-LaViRA on four heterogeneous embodiments shown in Figure 4, an Agilex Cobot Magic bimanual wheeled platform, a Unitree G1 humanoid, a Unitree Go1 quadruped, and a self-built quadrotor UAV. These four robots span three task families: VLN-CE, EQA, and ObjectNav. They also span three environment types: indoor office, indoor hall, and outdoor wild. No per-platform retraining is performed. The Language and Vision Action Models are byte-identical across the three ground robots, and only the low-level controller and the inference target change per platform. Ground robots run the MLLMs locally, while the UAV routes both calls through a remote API stack because its thermal and weight budgets rule out on-board inference at the required capacity.

A. Per-Embodiment Setup

For the three ground robots we replace the simulation-time FMM planner with NavDP [78] as the Robot Action stage. NavDP turns the current RGB-D observation into a short trajectory in the robot frame, which the platform's native velocity controller tracks. The UAV uses the PX4 position-control loop instead, because flight requires three-dimensional waypoint commands rather than planar velocity. Sensor calibration is the only platform-specific tuning step, and a hand–eye solve plus a checkerboard alignment of the body-fixed cameras converge in under fifteen minutes per platform.

Agilex Cobot Magic (wheeled bimanual). A chest-mounted Intel RealSense D435i RGB-D camera sits on a 2-DoF differential base, inherited from LaViRA [15]. NavDP trajectories are tracked by a proportional velocity controller. An onboard RTX 4090 serves a single Qwen3.5-27B-Q4 model for both LA and VA with no remote API call during operation.

Unitree G1 (humanoid). Four Orbbec Gemini 336L RGB-D cameras supply the four-view panorama, NavDP trajectories are executed by the native locomotion controller at (v_x, v_y, ω_z) sampled at 1 Hz with no gait re-training, and an onboard Jetson Orin NX runs Qwen3.5-9B-Q4 for both LA and VA.

Unitree Go1 (quadruped). Four Orbbec Gemini 336L RGB-D cameras supply the four-view panorama, sharing the same Jetson Orin NX and Qwen3.5-9B-Q4 stack as the Unitree G1. NavDP trajectories are tracked by the native velocity-mode controller. The quadruped is the most agile of the four platforms and executes wide turns without re-planning.

Self-built UAV (quadrotor). An Intel RealSense D435i matching the OpenUAV simulation provides RGB-D, and a Jetson Orin NX handles sensor I/O. The PX4 controller consumes (x, y, z) waypoint commands produced by NavDP from the depth observation and the target waypoint. LA runs on Gemini-3.1-Pro and VA on Qwen3.5-27B through a remote API stack. The single largest engineering surprise here was network jitter. A single LA call takes 30–60 s when the API is healthy and up to 150 s under contention, so we extend the plan horizon to roughly 5 m per call. Shorter horizons leave the UAV in a stale plan by the time the next observation lands, which we saw as occasional hover-and-redecide loops on the first deployment day.

B. Trials and Engineering Cost

We evaluate in an indoor office of $\sim$400 m^2, an outdoor corridor of 60 m, and an outdoor wild scene (Fig. 4). Tasks cover three simulation families with example instructions "Go to the chair on your left, turn right, go to the TV monitor and stop." for instruction following, "Find the bike." for open-vocabulary object finding, and "What is the color of the sofa in front of the gym?" for EQA. Real-world rollouts are run as qualitative validation rather than a quantitative benchmark, since simulation already provides a controlled setting for systematic metrics. UAV failures are dominated by altitude-induced visual ambiguity where the agent reaches

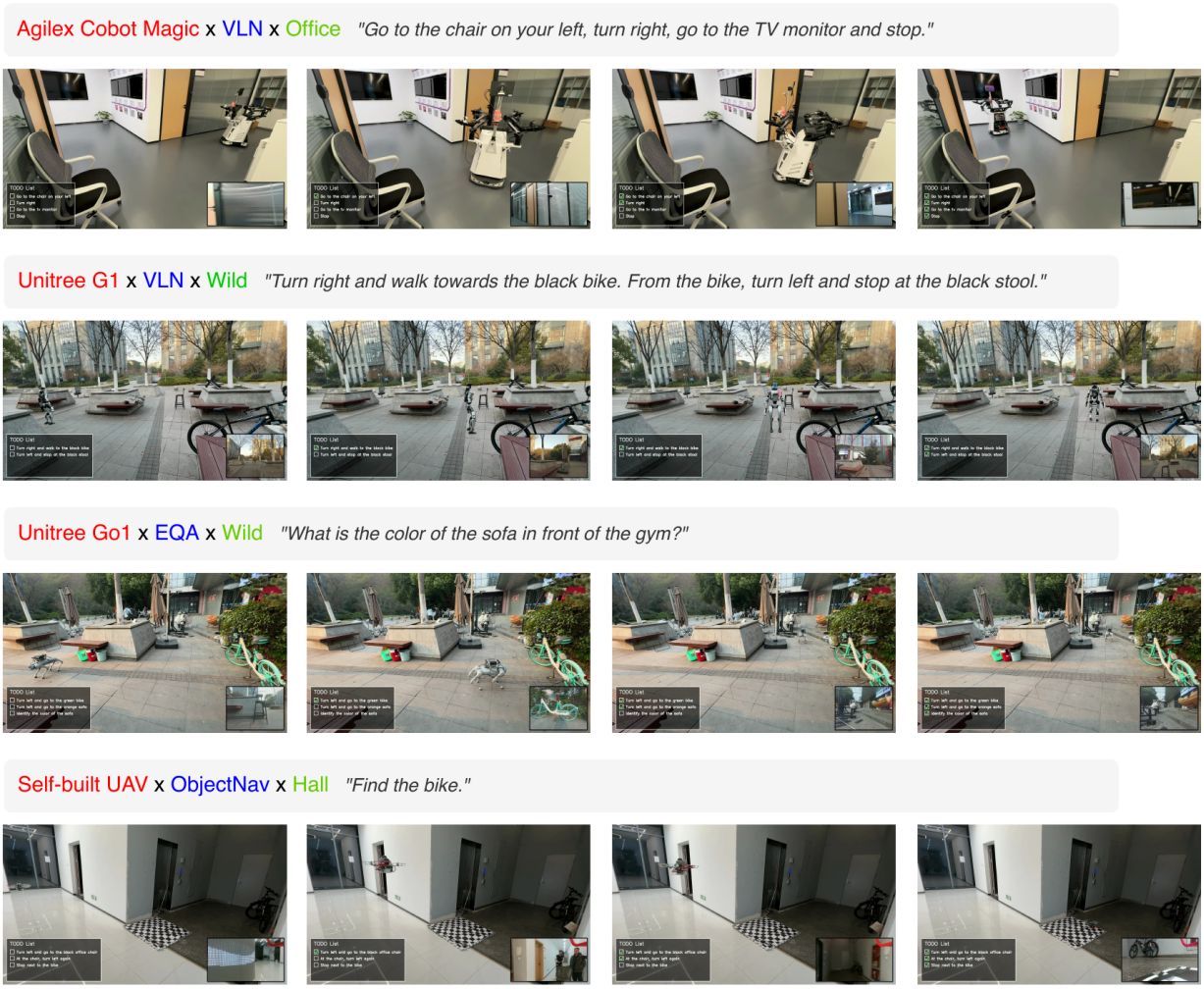


Fig. 4. **Real-world deployment across four embodiments, three task types, and three environments.** An Agilex Cobot Magic bimanual wheeled platform, a Unitree G1 humanoid, a Unitree Go1 quadruped, and a self-built quadrotor UAV share byte-identical Language and Vision Action Models across the three ground robots; only the low-level controller and the inference target change per platform.

the goal vicinity but cannot confirm the target from above and circles instead. The same verbatim panoramic prompt drives the Agilex Cobot Magic, Unitree G1, and the Unitree Go1, without edits, which we read as direct evidence that the decomposition abstracts away the viewpoint.

Engineering effort per embodiment. Adding a new embodiment takes only sensor calibration in $\sim$2 h, a velocity-controller adapter that maps 2-D waypoints to the platform's native command interface in $\sim$4–6 h with a 3-D extension for the UAV, and a brief integration test in $\sim$2 h. Total engineering effort across all four embodiments was roughly 40 person-hours, against the hundreds of GPU-hours typically required to retrain a VLA per platform. The Language and Vision Action Models are never modified.

Field-only failure modes. Three failure modes appear in the real world but not in simulation. *Lighting drift* shifts Vision Action confidences enough that target colours occasionally flip on object-finding episodes, mitigated by an exposure-locked frame at the start of each episode. *Network jitter* on the UAV triggers PX4's safety hover whenever a single LA call exceeds 150 s, costing roughly one extra step per affected episode. *Calibration drift* of $\sim$2 cm after thirty minutes of walking makes the Unitree G1 take overly wide turns at narrow doorways, fixed by a quick recalibration between task batches.

VI. Conclusion

We presented Uni-LaViRA, a training-free agentic framework that factors embodied navigation into three levels. The Language Action Model emits semantic-level directional commands, the Vision Action Model emits pixel-level visual targets, and the Robot Action controller back-projects those targets into the robot's native action space. Two agent-loop mechanisms make this unification practical. TODO List Memory keeps the agent's plan explicit by rewriting a checklist of

pending sub-goals at every step. Second Chance Backtrack rolls the robot back to its pre-error state and conditions the next plan on the failed sub-trajectory.

A single zero-shot implementation reaches 60.7% SR on VLN-CE R2R, 51.3% on VLN-CE RxR, 77.7% on HM3D-v2, 60.0% on HM3D-OVON, 54.7% on MP3D-EQA, and 40.00% on OpenUAV, matching or even surpassing trained foundation models that consume millions of samples and thousands of GPU-hours. The same agentic core deploys to four heterogeneous real robots, namely an Agilex Cobot Magic bimanual wheeled platform, a Unitree Go1 quadruped, a Unitree G1 humanoid, and a self-built quadrotor UAV, by swapping only the low-level controller.

Limitation and Future Work. Four limitations remain that map naturally onto the next steps. (i) The strongest backbones such as Gemini-3.1-Pro are proprietary, and open-weight alternatives still trail by a clear margin. We will distil agent traces from the proprietary Language and Vision Action Models into open backbones so the full pipeline can be self-hosted. (ii) Large-area grounding is unreliable, since "the hallway" is harder to localise than "a sofa". We will let the Vision Action Model call SAM [79] and Grounding DINO [80] as auxiliary tools when its bounding-box confidence is low, rather than as replacements. (iii) Long-instruction tasks such as RxR still leave a clear gap to trained VLA models, the dominant remaining limitation of a training-free design. We will extend TDM with hierarchical sub-goal compression and longer-context backbones to sustain attention over hundreds of words. (iv) Dynamic obstacles and humans are handled only by reactive low-level control. Lifting pedestrian-intent reasoning into the Language Action Model is the path to true social navigation, and is a direct extension of the TDM and SCB loop rather than a new architecture.

REFERENCES

[1] P. Anderson, Q. Wu, D. Teney, J. Bruce, M. Johnson, N. Sünderhauf, I. Reid, S. Gould, and A. van den Hengel, "Vision-and-language navigation: Interpreting visually-grounded navigation instructions in real environments," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 3674–3683.

[2] J. Krantz, E. Wijmans, A. Majumdar, D. Batra, and S. Lee, "Beyond the nav-graph: Vision-and-language navigation in continuous environments," in *Proc. European Conf. Computer Vision (ECCV)*, 2020, pp. 104–120.

[3] A. Ku, P. Anderson, R. Patel, E. Ie, and J. Baldridge, "Room-across-room: Multilingual vision-and-language navigation with dense spatiotemporal grounding," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2020.

[4] D. Batra, A. Gokaslan, A. Kembhavi, O. Maksymets, R. Mottaghi, M. Savva, A. Toshev, and E. Wijmans, "ObjectNav revisited: On evaluation of embodied agents navigating to objects," in *arXiv preprint arXiv:2006.13171*, 2020.

[5] S. K. Ramakrishnan, A. Gokaslan, E. Wijmans, O. Maksymets, A. Clegg, J. Turner, E. Undersander, W. Galuba, A. Westbury, A. X. Chang, M. Savva, Y. Zhao, and D. Batra, "Habitat-Matterport 3D dataset (HM3D): 1000 large-scale 3D environments for embodied AI," in *NeurIPS Datasets and Benchmarks*, 2021.

[6] N. Yokoyama, R. Ramrakhya, A. Das, D. Batra, and S. Ha, "HM3D-OVON: A dataset and benchmark for open-vocabulary object goal navigation," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2024.

[7] A. Das, S. Datta, G. Gkioxari, S. Lee, D. Parikh, and D. Batra, "Embodied question answering," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018.

[8] A. Majumdar, A. Ajay, X. Zhang, P. Putta, S. Yenamandra, M. Henaff, S. Silwal, P. Mcvay, O. Maksymets, S. Arnaud, K. Yadav, Q. Li, B. Newman, M. Sharma, V. Berges, S. Li, P. Jain, Y. Bisk, D. Batra, M. Kalakrishnan, F. Meier, C. Paxton, S. Sax, and A. Rajeswaran, "OpenEQA: Embodied question answering in the era of foundation models," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2024.

[9] X. Wang, D. Yang, Z. Wang, H. Kwan, J. Chen, W. Wu, H. Li, Y. Liao, and S. Liu, "Towards realistic UAV vision-language navigation: Platform, benchmark, and methodology," in *arXiv preprint arXiv:2410.07087*, 2024.

[10] J. Zhang, K. Wang, R. Xu, G. Zhou, Y. Hong, X. Fang, Q. Wu, Z. Zhang, and H. Wang, "NaVid: Video-based VLM plans the next step for vision-and-language navigation," in *Proc. Robotics: Science and Systems (RSS)*, 2024.

[11] J. Zhang, K. Wang, S. Wang, M. Li, H. Liu, S. Wei, Z. Wang, Z. Zhang, and H. Wang, "Uni-NaVid: A video-based vision-language-action model for unifying embodied navigation tasks," in *arXiv preprint arXiv:2412.06224*, 2024.

[12] A.-C. Cheng, Y. Ji, Z. Yang, X. Zou, J. Kautz, E. Bıyık, H. Yin, S. Liu, and X. Wang, "NaVILA: Legged robot vision-language-action model for navigation," in *arXiv preprint arXiv:2412.04453*, 2024.

[13] S. Wei, Z. Wang, Y. Zhang, J. Zhang, and H. Wang, "StreamVLN: Streaming vision-and-language navigation via SlowFast context modeling," in *arXiv preprint arXiv:2507.05240*, 2025.

[14] J. Zhang, Y. Zhang, K. Wang, S. Wei, Z. Wang, and H. Wang, "NavFoM: Towards a navigation foundation model for unified embodied navigation," in *arXiv preprint arXiv:2509.12129*, 2025.

[15] H. Ding, Z. Xu, Y. Fang, Y. Wu, Z. Chen, J. Shi, J. Huo, Y. Zhang, and Y. Gao, "LaViRA: Language-vision-robot actions translation for zero-shot vision language navigation in continuous environments," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2026.

[16] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, L. X. Shi, J. Tanner, Q. Vuong, A. Walling, H. Wang, and U. Zhilinsky, "π_0: A vision-language-action flow model for general robot control," in *arXiv preprint arXiv:2410.24164*, 2024.

[17] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn, "OpenVLA: An open-source vision-language-action model," in *CoRL*, 2024.

[18] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2023.

[19] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *NeurIPS*, 2023.

[20] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar *et al.*, "Inner monologue: Embodied reasoning through planning with language models," in *CoRL*, 2022.

[21] X. Shi, Z. Zhao, K. Gao, L. Wang, L. Chen, Y. Wang, and M. Liu, "SmartWay: Enhanced waypoint prediction and backtracking for zero-shot vision-and-language navigation," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2025.

[22] A. Chang, A. Dai, T. Funkhouser, M. Halber, M. Nießner, M. Savva, S. Song, A. Zeng, and Y. Zhang, "Matterport3d: Learning from RGB-D data in indoor environments," in *International Conference on 3D Vision (3DV)*, 2017.

[23] Y. Hong, Q. Wu, Y. Qi, C. Rodriguez-Opazo, and S. Gould, "VLN-BERT: A recurrent vision-and-language BERT for navigation," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2021.

[24] S. Chen, P.-L. Guhur, M. Tapaswi, C. Schmid, and I. Laptev, "Think global, act local: Dual-scale graph transformer for vision-and-language navigation," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2022.

[25] Z. Wang, X. Li, J. Yang, Y. Liu, and S. Jiang, "GridMM: Grid memory map for vision-and-language navigation," in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2023.

[26] D. An, Y. Qi, Y. Li, Y. Huang, L. Wang, T. Tan, and J. Shao, "BEVBert: Multimodal map pre-training for language-guided navigation," in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2023.

[27] D. An, H. Wang, W. Wang, Z. Wang, Y. Huang, K. He, and L. Wang, "ETPNav: Evolving topological planning for vision-language navigation in continuous environments," *IEEE Trans. Pattern Anal. Mach. Intell.*, 2024.

[28] Z. Wang, J. Li, Y. Hong, Y. Wang, Q. Wu, M. Bansal, S. Gould, H. Tan, and Y. Qiao, "Scaling data generation in vision-and-language navigation," in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2023.

[29] W. Zhu, H. Hu, J. Chen, Z. Deng, V. Jain, E. Ie, and F. Sha, "BabyWalk: Going farther in vision-and-language navigation by taking baby steps," in *Proc. Annu. Meeting Assoc. Computational Linguistics (ACL)*, 2020.

[30] X. Xue, J. Hu, M. Luo, S. Xie, J. Chen, Z. Xie, K. Quan, W. Guo, M. Xu, and Z. Chu, "OmniNav: A unified framework for prospective exploration and visual-language navigation," in *arXiv preprint arXiv:2509.25687*, 2025.

[31] Z. Chu, S. Xie, X. Wu, Y. Shen, M. Luo *et al.*, "ABot-N0: Technical report on the VLA foundation model for versatile embodied navigation," in *arXiv preprint arXiv:2602.11598*, 2026.

[32] Y. Qiao, W. Liu, J. Qin, T. Liu, B. Wang, and Q. Wu, "Open-Nav: Exploring zero-shot vision-and-language navigation in continuous environments with open-source LLMs," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2025.

[33] Y. Hong, Z. Wang, Q. Wu, and S. Gould, "Bridging the gap between learning in discrete and continuous environments for vision-and-language navigation," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2022.

[34] K. Chen, J. Fu, C. Gao, and N. Sang, "Constraint-aware zero-shot vision-language navigation in continuous environments," in *Proc. AAAI Conf. Artificial Intelligence*, 2025.

[35] Y. Long, W. Cai, H. Wang, G. Zhan, and H. Dong, "InstructNav: Zero-shot system for generic instruction navigation in unexplored environment," in *CoRL*, 2024.

[36] N. Yokoyama, S. Ha, D. Batra, J. Wang, and B. Bucher, "VLFM: Vision-language frontier maps for zero-shot semantic navigation," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2024.

[37] Z. Yin, H. Wang, Z. Cheng, Q. Chen, Y. Wang, and H. Dong, "GC-VLN: Instruction as graph constraints for training-free vision-and-language navigation," in *arXiv preprint arXiv:2509.22638*, 2025.

[38] K. Zhou, K. Zheng, C. Pryor, Y. Shen, H. Jin, L. Getoor, and X. E. Wang, "ESC: Exploration with soft commonsense constraints for zero-shot object navigation," in *Proc. Int. Conf. Machine Learning (ICML)*, 2023.

[39] Y. Kuang, H. Lin, and M. Jiang, "OpenFMNav: Towards open-set zero-shot object navigation via vision-language foundation models," in *arXiv preprint arXiv:2402.10670*, 2024.

[40] G. Zhou, Y. Hong, and Q. Wu, "NavGPT: Explicit reasoning in vision-and-language navigation with large language models," in *Proc. AAAI Conf. Artificial Intelligence*, 2024.

[41] J. Chen, B. Lin, R. Xu, Z. Chai, X. Liang, and K.-Y. K. Wong, "MapGPT: Map-guided prompting with adaptive path planning for vision-and-language navigation," in *Proc. Annu. Meeting Assoc. Computational Linguistics (ACL)*, 2024.

[42] Y. Long, X. Li, W. Cai, and H. Dong, "Discuss before moving: Visual language navigation via multi-expert discussions," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2024.

[43] M. Chang, T. Gervet, M. Khanna, S. Yenamandra, D. Shah, S. Y. Min, K. Shah, C. Paxton, S. Gupta, D. Batra, R. Mottaghi, J. Malik, and D. S. Chaplot, "GOAT: GO to any thing," in *Proc. Robotics: Science and Systems (RSS)*, 2024.

[44] N. Khanna, R. Yokoyama, D. Batra, A. Rai, and S. Ha, "GOAT-Bench: A benchmark for multi-modal lifelong navigation," in *CVPR*, 2024.

[45] S. Zeng, D. Qi, X. Chang, F. Xiong, S. Xie, X. Wu, S. Liang, M. Xu, X. Wei, and N. Guo, "JanusVLN: Decoupling semantics and spatiality with dual implicit memory for vision-language navigation," in *arXiv preprint arXiv:2509.22548*, 2025.

[46] M. Wei, C. Wan, J. Peng, X. Yu, Y. Yang, D. Feng, W. Cai, C. Zhu, T. Wang, J. Pang, and X. Liu, "Ground slow, move fast: A dual-system foundation model for generalizable vision-and-language navigation," in *arXiv preprint arXiv:2512.08186*, 2025.

[47] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog *et al.*, "Do as i can, not as i say: Grounding language in robotic affordances," in *CoRL*, 2022.

[48] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng, "Code as policies: Language model programs for embodied control," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2023.

[49] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, "Voyager: An open-ended embodied agent with large language models," *Trans. Mach. Learn. Res.*, 2024.

[50] S. Agashe, J. Han, S. Gan, J. Yang, A. Li, and X. E. Wang, "Agent-S: An open agentic framework that uses computers like a human," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2025.

[51] R. Bellman, "Dynamic programming," *Science*, vol. 153, no. 3731, pp. 34–37, 1966.

[52] R. S. Sutton, D. Precup, and S. Singh, "Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning," *Artificial Intelligence*, vol. 112, no. 1–2, pp. 181–211, 1999.

[53] X. Liu, J. Lin, Y. Hu, Y. Zhu, and R. Xu, "Adaptive zone-aware hierarchical planner for vision-language navigation," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2023.

[54] L. Zhang, Z. Zhang, W. Liu, H. Wang, Z. Lin, Y. Wang, and R. Xu, "NavA^3: Understanding any instruction, navigating anywhere, finding anything," in *arXiv preprint arXiv:2508.04598*, 2025.

[55] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, "MemGPT: Towards LLMs as operating systems," *arXiv preprint arXiv:2310.08560*, 2023.

[56] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proc. ACM Symposium on User Interface Software and Technology (UIST)*, 2023.

[57] W. Zhong, L. Guo, Q. Gao, H. Ye, and Y. Wang, "MemoryBank: Enhancing large language models with long-term memory," in *Proc. AAAI Conf. Artificial Intelligence*, 2024.

[58] M. Zhang, Y. Du, C. Wu, J. Zhou, Z. Qi, J. Ma, and B. Zhou, "ApexNav: An adaptive exploration strategy for zero-shot object navigation with target-centric semantic fusion," in *arXiv preprint arXiv:2504.14478*, 2025.

[59] Z. Zhu *et al.*, "Move to Understand a 3D Scene: Bridging visual grounding and exploration for efficient and versatile embodied navigation," in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2025.

[60] P. Xu, Z. Deng, J. Deng, Z. Gu, and S. Wan, "AerialVLA: A vision-language-action model for UAV navigation via minimalist end-to-end control," *arXiv preprint arXiv:2603.14363*, 2026.

[61] K. Yadav, R. Ramrakhya, S. K. Ramakrishnan, T. Gervet, J. Turner, A. Gokaslan, N. Maestre, A. Chang, D. Batra, M. Savva, A. X. Chang, and D. S. Chaplot, "Habitat-Matterport 3D semantics dataset," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2023.

[62] M. Savva, A. Kadian, O. Maksymets, Y. Zhao, E. Wijmans, B. Jain, J. Straub, J. Liu, V. Koltun, J. Malik, D. Parikh, and D. Batra, "Habitat: A platform for embodied AI research," in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2019.

[63] J. Liu, T. Xu, J. Chen, L. Yue, J. Zhang, Z. Wang, M. Li, Z. Zhang, and H. Wang, "SPAN-Nav: Generalized spatial awareness for versatile vision-language navigation," in *arXiv preprint arXiv:2603.09163*, 2026.

[64] K. Lyu, K. Wu, P. Li, X. Hu, Q. Si, C. Miao, N. Yang, Z. Wang, L. Xiao, L. Hu, J. Sun, and C. Hao, "HiMemVLN: Enhancing reliability of open-source zero-shot vision-and-language navigation with hierarchical memory system," *arXiv preprint arXiv:2603.14807*, 2026.

[65] W. Zheng, Y. Ge, and L. Itti, "Three-step nav: A hierarchical global-local planner for zero-shot vision-and-language navigation," *arXiv preprint arXiv:2604.26946*, 2026.

[66] W. Jiang, L. Wang, K. Huang, W. Fan, J. Liu, S. Liu, H. Duan, B. Xu, and X. Ji, "LongFly: Long-horizon UAV vision-and-language navigation with spatiotemporal context integration," *arXiv preprint arXiv:2512.22010*, 2025.

[67] W. Jiang, K. Huang, L. Wang, W. Xu, W. Fan, J. Liu, S. Liu, H. Liang, H. Duan, and B. Xu, "SpatialFly: Geometry-guided representation alignment for UAV vision-and-language navigation in urban environments," *arXiv preprint arXiv:2603.21046*, 2026.

[68] E. Wijmans, A. Kadian, A. Morcos, S. Lee, I. Essa, D. Parikh, M. Savva, and D. Batra, "DD-PPO: Learning near-perfect pointgoal navigators from 2.5 billion frames," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2020.

[69] R. Ramrakhya, E. Undersander, D. Batra, and A. Das, "Habitat-Web: Learning embodied object-search strategies from human demonstrations at scale," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2022.
[70] R. Ramrakhya, D. Batra, E. Wijmans, and A. Das, "PIRLNav: Pretraining with imitation and RL finetuning for ObjectNav," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2023.
[71] K. Yadav, A. Majumdar, R. Ramrakhya, N. Yokoyama, A. Baevski, Z. Kira, O. Maksymets, and D. Batra, "OVRL-V2: A simple state-of-art baseline for ImageNav and ObjectNav," in *arXiv preprint arXiv:2303.07798*, 2023.
[72] D. Zheng, S. Huang, L. Zhao, Y. Zhong, and L. Wang, "Towards learning a generalist model for embodied navigation," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2024.
[73] N. Yokoyama and S. Ha, "FiLM-Nav: Efficient and generalizable navigation via VLM fine-tuning," in *arXiv preprint arXiv:2509.16445*, 2025.
[74] B. Yu, H. Kasaei, and M. Cao, "L3MVN: Leveraging large language models for visual target navigation," in *IROS*, 2023.
[75] H. Yin, X. Xu, Z. Wu, J. Zhou, and J. Lu, "SG-Nav: Online 3D scene graph prompting for LLM-based zero-shot object navigation," in *NeurIPS*, 2024.
[76] W. An *et al.*, "DSCD-Nav: Dual-stance cooperative debate for object navigation," *arXiv preprint arXiv:2601.21409*, 2026.
[77] F. Wu *et al.*, "ReMemNav: A rethinking and memory-augmented framework for zero-shot object navigation," *arXiv preprint arXiv:2603.26788*, 2026.
[78] W. Cai, J. Peng, Y. Yang, Y. Zhang, M. Wei, H. Wang, Y. Chen, T. Wang, and J. Pang, "NavDP: Learning sim-to-real navigation diffusion policy with privileged information guidance," *arXiv preprint arXiv:2505.08712*, 2025.
[79] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. C. Berg, W.-Y. Lo, P. Dollár, and R. Girshick, "Segment anything," in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2023.
[80] S. Liu, Z. Zeng, T. Ren, F. Li, H. Zhang, J. Yang, C. Li, J. Yang, H. Su, J. Zhu, and L. Zhang, "Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection," in *Proc. European Conf. Computer Vision (ECCV)*, 2024.